

Task 1

To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands Update Input Buffer, Test Recall Functionality

```
root@Tejasri:~# nano session3.c
root@Tejasri:~# |
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 100
#define MAX_INPUT 1024

char history[MAX_HISTORY][MAX_INPUT];
int history_count = 0;
int history_pos = -1;

void add_history(const char *command) {
    if (strlen(command) == 0)
        return;

    if (history_count < MAX_HISTORY) {
        strcpy(history[history_count], command);
        history_count++;
    } else {
        for (int i = 1; i < MAX_HISTORY; i++)
            strcpy(history[i - 1], history[i]);

        strcpy(history[MAX_HISTORY - 1], command);
    }

    history_pos = history_count;
}

void show_history() {
```

```

    for (int i = 0; i < history_count; i++)
        printf("%d  %s\n", i + 1, history[i]);
}

void previous_command(char *buffer) {
    if (history_count == 0) {
        printf("\nNo command history.\n");
        return;
    }

    if (history_pos > 0)
        history_pos--;

    strcpy(buffer, history[history_pos]);
}

void next_command(char *buffer) {
    if (history_count == 0) {
        printf("\nNo command history.\n");
        return;
    }

    if (history_pos < history_count - 1) {
        history_pos++;
        strcpy(buffer, history[history_pos]);
    } else {
        history_pos = history_count;
    }
}

```

```
root@Tejasri:~# gcc session3.c -o session3
root@Tejasri:~# ./session3
Simple Shell History Test
Commands:
  history      - Show command history
  up           - Recall previous command
  down        - Recall next command
  exit         - Exit shell

myshell> pwd
Executed: pwd
myshell> history

Command History:
1  pwd
myshell> |
```

TASK 2

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind

```
root@Tejasri:~# nano Session3.c
root@Tejasri:~# |
```

```

GNU nano 7.2 Session3.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
/* Dynamic array */
typedef struct {
    int *data;
    int size;
    int capacity;
} DynamicArray;
/* Linked List Node */
typedef struct Node {
    int data;
    struct Node *next;
} Node;
/* Initialize dynamic array */
void initArray(DynamicArray *arr, int capacity) {
    arr->data = (int *)malloc(capacity * sizeof(int));
    arr->size = 0;
    arr->capacity = capacity;
    if (arr->data == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
}
/* Resize dynamic array */
void resizeArray(DynamicArray *arr, int newCapacity) {
    int *temp = (int *)realloc(arr->data, newCapacity * sizeof(int));
    if (temp == NULL) {
        printf("Array resizing failed!\n");
        return;
    }
    arr->data = temp;
    arr->capacity = newCapacity;
}
/* Insert into dynamic array */
void insertArray(DynamicArray *arr, int value) {
    if (arr->size == arr->capacity) {
        resizeArray(arr, arr->capacity * 2);
    }
    arr->data[arr->size++] = value;
}

```

OUTPUT:

```

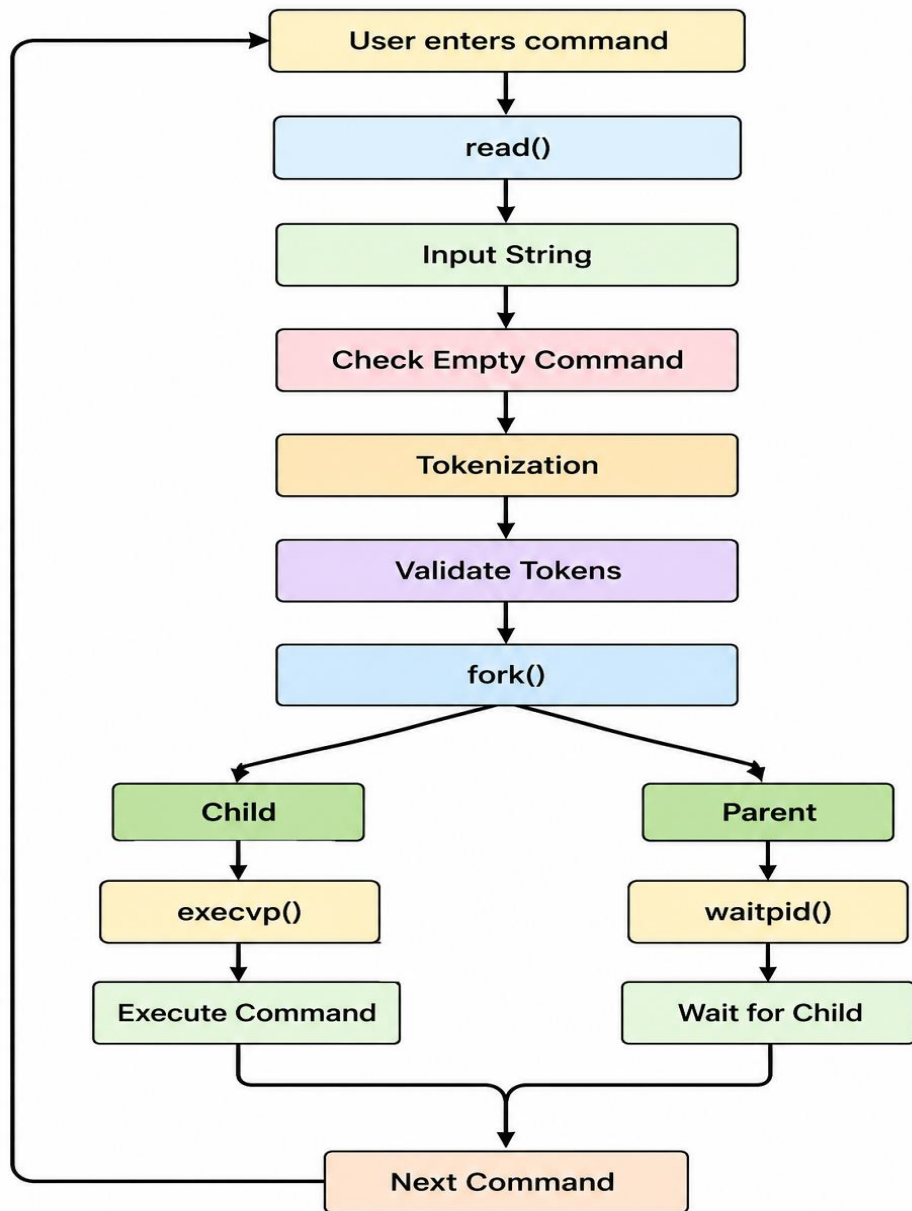
root@tejasri:~# gcc Session3.c -o Session3
root@Tejasri:~# ./Session3
Dynamic Array: 10 20 30 40
Linked List: 300 -> 200 -> 100 -> NULL
Memory released successfully.
root@Tejasri:~# |

```

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

Flow Chart:



```

GNU nano 7.2 mys
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_INPUT 1024
#define MAX_TOKENS 100
int main() {
    char input[MAX_INPUT];
    while (1) {
        printf("myShell> ");
        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';
        // Handle empty command
        if (strlen(input) == 0)
            continue;
        char *args[MAX_TOKENS];
        int count = 0;
        // Tokenization
        char *token = strtok(input, " \t");
        while (token != NULL && count < MAX_TOKENS - 1) {
            args[count++] = token;
            token = strtok(NULL, " \t");
        }
        args[count] = NULL;
        // Display parsed arguments
        printf("--- Parsed Arguments ---\n");
        for (int i = 0; i < count; i++) {
            printf("Argument %d: %s\n", i + 1, args[i]);
        }
        // Exit command
        if (strcmp(args[0], "exit") == 0)
            break;
        // Create child process
        pid_t pid = fork();
        if (pid < 0) {
            perror("fork failed");
            continue;
        }
        if (pid == 0)

```

```
root@Tejasri:~# nano MyShell.c
root@Tejasri:~# gcc MyShell.c -o MyShell
root@Tejasri:~# /
-bash: /: Is a directory
root@Tejasri:~# ./MyShell
myShell> echo Hello
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello
Child PID: 609
Hello
Command execution completed.
myShell> |
```

TASK 2

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

```
root@Tejasri:~# gcc Task2.c -o Task2
root@Tejasri:~# ./Task2
Enter command: ls

Command: ls
Number of arguments: 0

Parse Tree:
COMMAND
|
+-- ls

Execution structure:
args[0] = ls
root@Tejasri:~# |
```

OUTPUT:

```
root@Tejasri:~# nano Task2.c
root@Tejasri:~# |
```


GNU nano 7.2

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#define MAX 100
typedef struct {
    char command[MAX];
    char *args[MAX];
    int argc;
} Command;

void parseCommand(char *input, Command *cmd) {
    cmd->argc = 0;
    char *token = strtok(input, " \\t\\n");
    while (token != NULL && cmd->argc < MAX - 1) {
        cmd->args[cmd->argc++] = token;
        token = strtok(NULL, " \\t\\n");
    }
    cmd->args[cmd->argc] = NULL;
    if (cmd->argc > 0) {
        strcpy(cmd->command, cmd->args[0]);
    } else {
        cmd->command[0] = '\\0';
    }
}

int validateSyntax(Command *cmd) {
    if (cmd->argc == 0)
        return 0; // Empty command
    // Basic syntax validation
    for (int i = 0; i < cmd->argc; i++) {
        if (cmd->args[i][0] == '\\0')
            return -1;
    }
    return 1;
}

void printParseTree(Command *cmd) {
    printf("\\nParse Tree:\\n");
    printf("COMMAND\\n");
}
```